



Instructor: Date:

Learner: Regional Branch:

Program: Center:

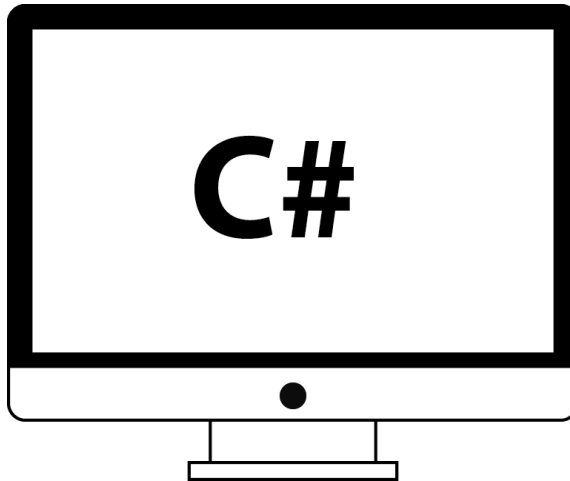
CONTENT

- Let's Start! • Let's Explore! • Let's Practice! • Extension Activity • Target Words • Learning Points



Let's Start!

Let's learn some constants in C#.



Learning Outcome:

Talk about principles, concepts, characteristics of general-purpose computer programming language in spoken and written situations.

Ask your partner:

- What is a variable in C#?
- Do you know what is the opposite of a variable? What are some examples?

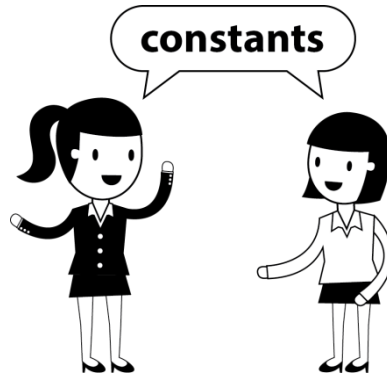
Can Do Statements:

- I can talk about the concept of constants in C#.
- I can list different kinds of constants in C#.
- I can talk about characteristics of different kinds of constants in C#.



Let's Explore!

Ruth & Baxter are going to prepare a presentation about a topic. Can you tell what the topic is about?



Instructions: Read the text below.

- Wolfgang:** Hey, Baxter! Come here a second. This is Ruth. She's a programmer from our Vermont branch. You two are going to have to create a mini presentation on the constants of C# for the next training session; don't worry, you'll be paid overtime.
- Ruth:** Hi, Baxter.
- Baxter:** Hi, Ruth.
- Wolfgang:** I'll leave you two to it. Let me know if you need anything.
- Ruth & Baxter:** Yes, sir!
- Ruth:** Ok, so this should be pretty simple. In the first slide, we should explain that constants are sometimes also called literals and that they are fixed values that the programmer may not alter during its execution.
- Baxter:** Correct. Then we're going to have to break down the different types of Integer literals, floating-point literals, character constants, and string literals; just a brief explanation should suffice.
- Ruth:** Right. So, we may want to say that integer literals can be a decimal or a hexadecimal constant.
- Baxter:** You have got to talk about the prefix and the suffix as well. Maybe we can put in a different slide with bright colors: "Prefix specifies the 0x or 0X for hexadecimal. No prefix id for decimal." In the second slide, we can project, "Suffix: a combination of U (unsigned) and L (long); the suffix is not case sensitive."
- Ruth:** Great idea! When we talk about floating-point, we are going to have to include that it has an integer part, a decimal point, a fractional part, and an exponent part.
- Baxter:** I believe that we should hand out a photocopy with all the different combinations of characters. However, during the presentation, we should highlight that characters are enclosed in single quotes. We must name at least three categories: universal character, escape sequence, and plain character. They'll have the explanation on paper.



Let's Practice!

Can you match the types of constants?

Instructions: Match each word with a definition.

COLUMN A

1. Constant
2. Literals
3. integer literals
4. Floating point
5. character constants
6. string literals

COLUMN B

- a. Literals that may be within double quotes or preceded by an @
- b. Fixed value that can be repeated throughout the coding process
- c. Literals that can include an integer part, decimal point and functional part represented by a decimal or exponential form.
- d. Decimal or Hexadecimal constant can be coupled with a prefix
- e. Fixed and unchangeable value
- f. Special characters that may represent constants that are preceded by backslash for example: \\



Let's Practice!

Can you find the difference between Constants and Literals?

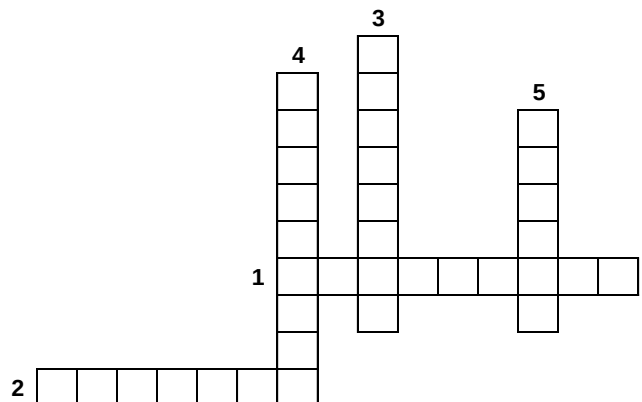
Instructions: Read the clues. Complete the crossword.

Down

1. If you are going to have to code an app where you need unchangeable and unfixed lines of code you need to use _____
2. You've got to be talking about _____ literals since you are mentioning hexadecimal constants.

Across

3. You are going to have to understand that _____ point literals have integer parts and decimal points.
4. You've got to know that _____ literals are sometimes represented by Unicode or escape sequences such as "\\n" to refer to a newline.
5. The following line of code is a _____ literal as it is enclosed by quotes "hello \ world"





Extension Activity

How much do you think you know about C#?

Instructions: Complete the following steps. Fill in the Self-Assessment.

- Do further research about C# constants and literals and the differences between C and C++.
- Write down those differences individually by using "can" and "may" to say what things are possible with each one.
- Create two short paragraphs comparing C and C++

Self-Assessment

Yes

No

- I research on specific topics.
- I list differences between C, C++ and C# while making use of grammar topics such as can and may for possibility.
- I explain differences among several items.

☐ ☐
☐ ☐
☐ ☐


Target Words

- **break down** (verb): to divide into sections
e.g. Then we're going to have to break down the different types of constants.
- **should suffice** (expression): should be enough.
e.g. Just a brief explanation should suffice.
- **hand out** (phrasal verb): to give something to a group, such as information or material
e.g. I believe that we should hand out a photocopy with all the different combinations of characters.
- **highlight** (verb): to draw attention.
e.g. We should highlight that characters are enclosed in single quotes
- **single quotes** (noun): one pair of single quotation marks
e.g. However, during the presentation, we should highlight that characters are enclosed in single quotes.



Learning Points

May and Can to Talk About Possibility

To express possibility, we can use the modal verbs **can** and **may**; they have different degree of certainty.

We use **can** to talk about a general possibility; something that is likely to happen frequently.

Form: Subject + **can** + **base form of the verb** + complement

Let's understand a little bit more by checking some examples:

*Programming **can be** very hard at the beginning.* (There is the possibility of finding programming hard at the beginning, because several other people have experienced it that way.

*You **can fail** the test on C# if you don't study hard enough.* (Very often, when people don't study, they fail their tests).

We use **may** to express the possibility of something to happen, either in the present or in the future.

Form: Subject + **may** + **verb** complement

Let's understand a little bit more by checking some examples:

*It **may** rain tomorrow!* (There exists the possibility of a rain the day after.

*Carla brought her car today; she **may give** you a ride.* (There exists the possibility of Carla taking the person to another place).

Examples

- *Constants of C# are fixed values that the programmer may not alter during its execution*
- *So, we may want to say that integer literals can be a decimal or a hexadecimal constant.*
- *Maybe we can put in a different slide with bright colors..*





Learning Points

Going to Have to for What Is Necessary

We use **going to have to** to talk about things that become, at certain point, obligations.

We use **going to have to** with two purposes mainly.

Structure: Subject + verb to be + **going to have to** + infinitive + complement.

To say that someone should do something:

Example: *You're **going to have to** code a new program tomorrow.*

You can also use this structure to talk about things you didn't want to do, but now they have become an obligation.

Example: *I'm **going to have to** start the code programming again; there's a mistake from the very core of it.*

This structure is not so common in written English, but it is so in colloquial English. However, native speakers always pronounce **going to** in the contracted form **gonna**.

Examples

- *You have got to talk about the prefix and the suffix as well.*
- *We are going to have to include that it has an integer part, a decimal point, a fractional part, and an exponent part.*